Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 2

Дисциплина МДК 03.02

Гармонизация международных и национальных стандартов

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Основания сравнения.....3

1.2 Термины и этапы испытаний.....3

1.3 Адаптация документации.....4

1.4 Выводы.....5

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....6

ПРИЛОЖЕНИЕ А Листинг исходного кода.....7

1 ВЫПОЛНЕНИЕ РАБОТЫ

Я выполнил работу по варианту 2 Library System АИС «Умный кампус» на C# и .NET 8. Модуль принимает PDF/EPUB/ZIP в карантин; права Reader и Librarian разделены [6]. Оформление выполнено по положению института [5].

1.1 Основания сравнения

Я сопоставил отечественный подход к испытаниям и документации с процессами ISO/IEC/IEEE 12207:2026. ГОСТ 34.603-92 рассмотрен как документ, прямо названный в задании. Он утратил силу в РФ; для применения на территории РФ указано пользоваться ГОСТ Р 59792-2021. Поэтому историческое сравнение не означает признания старого документа действующим [3] [4] [7].

Документы ЕСПД разделяют ТЗ и программу испытаний. Первое фиксирует требования и порядок приёмки, второе — условия, действия и ожидаемый результат. ГОСТ 19.201-78 регулирует состав ТЗ, а не стиль C# [1] [2].

1.2 Термины и этапы испытаний

Предварительные испытания, опытная эксплуатация и приёмочные испытания АС не являются синонимами unit, integration и CI. Автоматический тест предоставляет свидетельство выполнения требования, но сам по себе не заменяет опытную эксплуатацию и решение о приёмке [4].

Верификация проверяет соответствие установленным требованиям, валидация — пригодность для потребностей пользователя. Например, тест ZIP с ../escape.pdf проверяет FR-03; сценарий библиотекаря «загрузить книгу и увидеть карантин» позволяет оценить пригодность процесса. Эти виды контроля относятся к жизненному циклу, а не только к последнему дню разработки [7].

Сведения представлены в таблице 1.

Таблица 1 — Гармонизация требований

Область	Национальные документы	Процессный подход	Артефакт
Термины	Предварительные, опытная эксплуатация, приёмочные	Verification и validation	Требование ↔ тест ↔ сценарий
Планирование	ТЗ и программа испытаний ЕСПД	Процессы контроля на всём ЖЦ	technical-assignment.md и test-plan.md
Приём загрузки	Действие и ожидаемый результат	Объективные свидетельства требований	201, 400, 401, 403, 413, 429
Документы	Протокол и перечень отклонений	Управляемые записи результатов	TRX, JSON, баг-репорты
Изменение	Повторные испытания после исправления	Конфигурация и верификация изменений	before, Git, регрессионные тесты
Приёмка	Решение по ТЗ	Подтверждение потребностей	Сценарий библиотекаря; тест не заменяет решение о приёмке

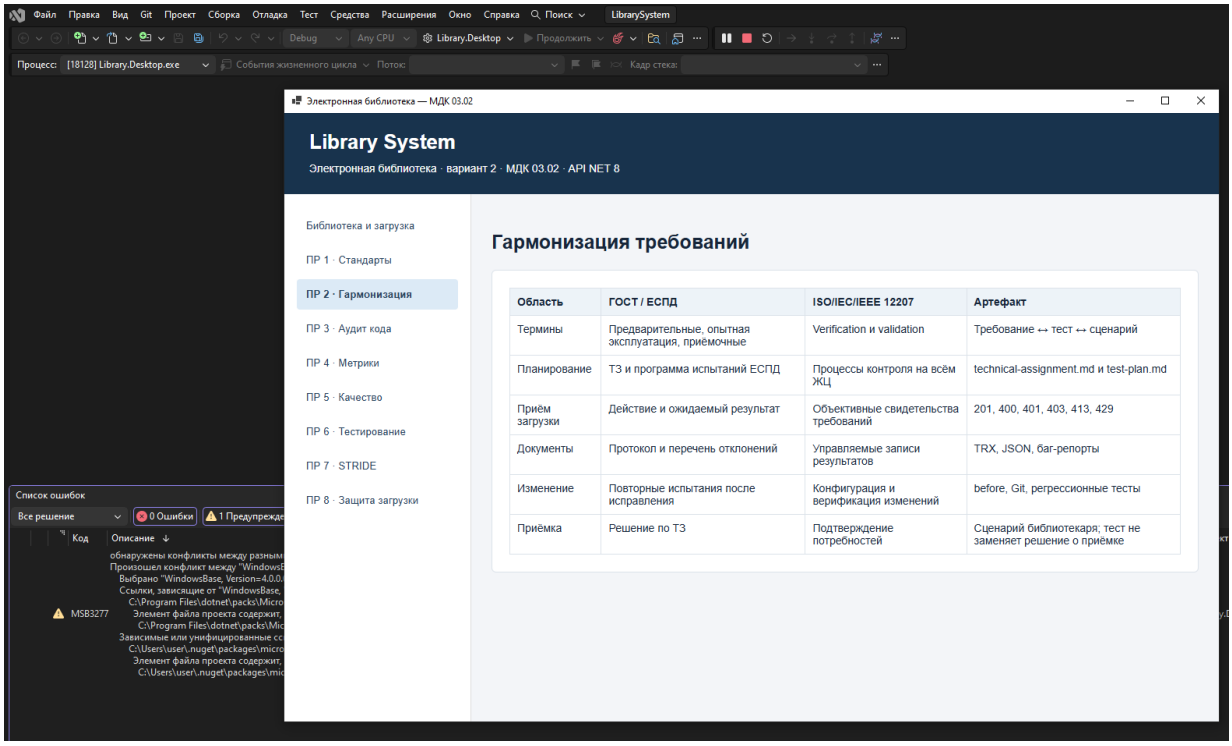
Источник: составлено автором на основе [3], [4], [7], [1], [2]

1.3 Адаптация документации

Я сохранил отдельные LS-TZ-01 и LS-TP-01. Каждый FR имеет критерий и имена тестов в traceability.json. Исходный TRX хранит результаты, JSON используется для просмотра, отчёт фиксирует условия и ограничения. Состав тестовых случаев и записей о дефектах согласован с ISO/IEC/IEEE 29119-3:2021 [8].

Приёмка загрузки выражена наблюдаемыми признаками: PDF/EPUB — 201 и Quarantined; Reader — 403; без токена — 401; опасный ZIP — 400 без сохранения файлов; четвёртая загрузка — 429. После изменения кода выполняется регрессия, а версия документов обновляется вместе с исходниками. Команда harmonization создаёт таблицу JSON/CSV.

В окне программы показана сравнительная таблица требований и артефактов (рисунок 1).



Источник: снимок прогона на Windows, предоставленный автором.

Рисунок 1 — Таблица гармонизации ГОСТ и ISO/IEC/IEEE 12207 в приложении

1.4 Выводы

Я разработал таблицу соответствий без искусственного равенства отдельных стадий. Адаптация сохраняет отечественный комплект документов и связывает его с процессами, требованиями и объективными результатами проверок.

Полные исходники, входные данные и журналы переданы с отчётом. Основной листинг приведён в приложении А.

Исходный код и порядок запуска: репозиторий MDK0302-LibrarySystem.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78. ЕСПД. Техническое задание. Требования к содержанию и оформлению. — URL: https://protect.gost.ru/gost/details/b64061cf-6bb2-4c75-aee9-18bb5e26e554 (дата обращения: 08.11.2026) [1].

2. ГОСТ 19.301-79. ЕСПД. Программа и методика испытаний. Требования к содержанию и оформлению. — URL: https://protect.gost.ru/gost/details/b25e0645-091a-425b-8558-3e0c3ce37315 (дата обращения: 08.11.2026) [2].

3. Росстандарт. ГОСТ 34.603-92. Карточка документа и сведения о прекращении применения в РФ. — URL: https://protect.gost.ru/gost/details/509c908a-b61f-4969-a998-f3e684700638 (дата обращения: 08.11.2026) [3].

4. ГОСТ Р 59792-2021. Информационные технологии. Виды испытаний автоматизированных систем. — URL: https://protect.gost.ru/gost/details/60279c95-21dc-430c-bf2c-f626c588576e (дата обращения: 08.11.2026) [4].

5. Положение об оформлении письменных работ. Санкт-Петербургский институт экономики и управления. Методические материалы [5].

6. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [6].

7. ISO/IEC/IEEE 12207:2026. Systems and software engineering. Software life cycle processes. — URL: https://www.iso.org/standard/90219.html (дата обращения: 08.11.2026) [7].

8. ISO/IEC/IEEE 29119-3:2021. Software testing. Part 3. Test documentation. — URL: https://www.iso.org/standard/79429.html (дата обращения: 08.11.2026) [8].

ПРИЛОЖЕНИЕ А
Листинг исходного кода

src/Library.Tools/Program.cs

```
using System.Diagnostics;
using System.Text;
using System.Text.Json;
using System.Text.Encodings.Web;
using System.Xml.Linq;
using Library.Tools;
using Library.Core;
Console.OutputEncoding = Encoding.UTF8;
var directory = new DirectoryInfo(Directory.GetCurrentDirectory());
while (directory is not null && !Directory.Exists(Path.Combine(directory.FullName, "artifacts"))) directory =
directory.Parent;
string root = directory?.FullName ?? throw new IOException("Запуск из корня Project");
string verification = Path.Combine(root, "verification");Directory.CreateDirectory(verification);
var options = new JsonSerializerOptions(JsonSerializerDefaults.Web) { WriteIndented = true, Encoder =
JavaScriptEncoder.UnsafeRelaxedJsonEscaping };
void Save(string name, object data) => File.WriteAllText(Path.Combine(verification, name + ".json"),
JsonSerializer.Serialize(data, options));
string command = args.FirstOrDefault() ?? "all";
try
{
  if (command is "all" or "samples")
  {
    Directory.CreateDirectory(Path.Combine(root, "samples"));
    File.WriteAllBytes(Path.Combine(root, "samples/sample.pdf"), Samples.Pdf());
    File.WriteAllBytes(Path.Combine(root, "samples/sample.epub"), Samples.Epub());
    File.WriteAllBytes(Path.Combine(root, "samples/zip-slip.zip"), Samples.Zip(("../escape.pdf",
Samples.Pdf())));
  }
  foreach (string name in new[] { "standards", "harmonization", "quality", "threats", "bugs" })
    if (command == "all" || command == name)
    {
      string text = File.ReadAllText(Path.Combine(root, "artifacts", name + ".json"));
      File.WriteAllText(Path.Combine(verification, name + ".json"), text);
      var rows = JsonDocument.Parse(text).RootElement;
      string Escape(string s) => "\"" + s.Replace("\"", "\"\"") + "\"";
      var lines = new List<string> { string.Join(';', rows[0].EnumerateObject().Select(x => Escape(x.Name))) };
      lines.AddRange(rows.EnumerateArray().Select(row => string.Join(';', row.EnumerateObject().Select(x
=> Escape(x.Value.ToString())))));
      File.WriteAllLines(Path.Combine(verification, name + ".csv"), lines, new UTF8Encoding(true));
      Console.WriteLine(name + ": строк " + rows.GetArrayLength());
    }
  if (command is "all" or "audit" or "metrics")
  {
    string before = File.ReadAllText(Path.Combine(root, "before/LegacyUploadController.cs"));
    string after = File.ReadAllText(Path.Combine(root, "src/Library.Api/UploadController.cs"));
    string catalog = File.ReadAllText(Path.Combine(root, "src/Library.Api/BooksController.cs"));
    var result = new { before = CodeQuality.Measure(before, "LegacyUploadController"),
      after = CodeQuality.Measure(after, "UploadController"), catalog = CodeQuality.Measure(catalog,
"BooksController"),
      beforeAudit = CodeQuality.Audit(before), afterAudit = CodeQuality.Audit(after), catalogAudit =
CodeQuality.Audit(catalog) };
    Save("analysis", result);Console.WriteLine(JsonSerializer.Serialize(result, options));
  }
```

```
  if (command == "parse-tests")
  {
    string path = args.Length > 1 ? args[1] : Path.Combine(verification, "library-tests.trx");
    var tests = XDocument.Load(path).Descendants().Where(x => x.Name.LocalName == "UnitTestResult")
      .Select(x => new { name = (string?)x.Attribute("testName"), outcome = (string?)x.Attribute("outcome"),
duration = (string?)x.Attribute("duration") }).OrderBy(x => x.name).ToArray();
    if (tests.Length == 0) throw new IOException("Нет результатов TRX");
    var result = new { total = tests.Length, passed = tests.Count(x => x.outcome == "Passed"), failed =
tests.Count(x => x.outcome == "Failed"), skipped = tests.Count(x => x.outcome is not ("Passed" or "Failed")),
tests };
    Save("tests", result);Console.WriteLine($"Всего {result.total}; Passed {result.passed}; Failed
{result.failed}; Skipped {result.skipped}");
    if (result.failed > 0 || result.skipped > 0) Environment.ExitCode = 1;
  }
  if (command == "api-checks")
  {
    using var client = new HttpClient { BaseAddress = new Uri(args.Length > 1 ? args[1] :
"http://127.0.0.1:5352") };
    using var response = await client.PostAsync("/lab/scenarios", null);response.EnsureSuccessStatusCode();
    string text = await response.Content.ReadAsStringAsync();File.WriteAllText(Path.Combine(verification,
"api-scenarios.json"), text);Console.WriteLine(text);
    if (JsonDocument.Parse(text).RootElement.EnumerateArray().Any(x => !
x.GetProperty("passed").GetBoolean())) Environment.ExitCode = 1;
  }
  if (command == "open-ui")
  {
    var url = new Uri(args[1]);using var client = new HttpClient { Timeout = TimeSpan.FromSeconds(2) };bool
ready = false;
    for (int i = 0; i < 30 && !ready; i++)
    {
      try { using var response = await client.GetAsync(new Uri(url, "/health"));ready =
response.IsSuccessStatusCode; }
      catch (HttpRequestException) { } catch (TaskCanceledException) { }
      if (!ready) await Task.Delay(500);
    }
    if (!ready) throw new IOException("Сервер ещё не запущен");
    Process.Start(new ProcessStartInfo(url.AbsoluteUri) { UseShellExecute = true });
  }
}
catch (Exception e) { Console.Error.WriteLine("Операция не выполнена: " +
e.Message);Environment.ExitCode = 1; }
```